STUDYROUTE / INTERNATIONAL TRACK / TOPIC C

Technical Documentation

Architecture, core algorithms, trained components and security design

SANDBOX / SYNTHETIC DATA / NOT PRODUCTION ASSURANCE

System description with measured evidence and stated limits

No real money, live bank, real users or independent certification

REPORT CONTENTS

00 Summary
01 System architecture
02 Core algorithms and trained components
03 Security design
04 The simulated financial layer
05 Verification and evidence
06 Limitations and next steps
07 Provenance, licences and AI assistance

SR-TEC-2026-01 | Final edition

18 October 2026

Prepared for StudyRoute submission review • English edition • Team TeamEkxin

00 / SUMMARY

Summary

StudyRoute is a sandbox agent that turns one sentence from a parent into a ranked, deadline-aware payment plan that only a person can approve. This report describes how it is built, how it was evaluated, and where the evidence stops.

Assessment area	Conclusion
Architecture	A pure engine computes every number. A domain layer enforces a state machine whose approval step needs a role, a token scope, ownership, a fresh MFA proof and the hash of the summary the person saw. Models sit behind deterministic checks and are optional.
Core algorithms	Exact integer-money cost ranking, a two-calendar deadline planner, a send-plan optimiser and a stress test, each asserted by tests.
Learned components	Four small models: a reader, a dialogue policy, an explainer and a review router. Each is measured against a baseline we ran, none is a payment authority, and the modest results are stated (sections 2.7 to 2.12).
Security	Human-only, MFA-bound approval; idempotency; signed webhooks; a hash-chained audit log; a 146-attack self-run probe. Self-assessed: no outside test.
Readiness	A controlled, synthetic-data sandbox demonstration. Not production assurance.

> **Not independently certified** — This is the team's own description and evidence. It is not an audit, a penetration test, a banking approval or a statement that no vulnerabilities exist. Sections 5 and 6 say what was checked and what was not.

The problem. A parent in Bangladesh paying tuition abroad usually compares banks by the fee they are shown. The larger cost is the exchange-rate margin. In our worked example (GBP 10,000, synthetic rates) 63% of the best provider's cost is margin: BDT 16,000 of BDT 25,300. Fees taken on the way can leave the university short-paid, the due date has to respect two calendars, and the paperwork is unclear. The corridor is large and growing: Bangladesh Bank data, reported by The Observer on 24 July 2026 [8], show USD 881.7 million sent through banks for overseas education in 2025-26, a record and 32% more than a year earlier.

What we built. StudyRoute is a sandbox agent. One sentence ("I need to pay GBP 10,000 tuition by 28 October") becomes a quote with every provider ranked by true total cost, a deadline plan across both countries' working days, a document checklist and a draft. The parent approves with a fresh MFA proof bound to the exact summary they saw. A simulated provider settles the payment, and every step is written to a hash-chained audit log. Around that core are a student budget and arrival guide, a simulated bank network, an escrow on a simulated chain, and a bank review desk that routes unusual payments to a person.

What we claim as our contribution. Nothing in the algorithms is new: decimal arithmetic, JWT, Argon2, HMAC, hash chains, ONNX, Double DQN and distillation are all established. The contribution is how they are put together and how well the result is checked.

1. *An architecture in which a language model can help without being able to hurt.* Every number comes from a pure engine. Each learned component has a deterministic check or a fallback around it. Only a person with a fresh MFA proof can approve a payment, and a permission table in code, not a prompt, enforces that.
2. *Four small trained models, each built for one job, each measured against a baseline we actually ran, each with its limits published:* a reader of untidy requests, a dialogue policy, an explainer that rewords the engine's answers, and a router that decides which payments a bank reviewer looks at first.
3. *Evidence a reviewer can re-run, and a list of what it does not show:* 1,679 tests mapped to 24 scenarios, a 146-attack black-box probe, a run in Docker, and a record of what the runs found, including a defect.

Evidence at a glance (sources in section 5 and in `submission/evidence/`):

What was measured	Result	Reads as
Cost ranking, worked example	B 1.58%, C 1.63% (university GBP 35 short), A 2.16%	Exact; asserted in `test_engine_quote.py`
Backend tests	1,679 (1,678 pass, 1 skipped on Windows)	One or more tests for each of 24 scenarios
Black-box attack probe	146 of 146 defended; real-browser audit 48 of 48	Written by the team, not independent
Reader against rules, 113 frozen hand-written messages	Intent 88.5% against 61.1%; plan read exactly 100% against 57.6%; attacks blocked 92.9% against 28.6%	Small sample, about plus or minus 7 points
Dialogue policy against plan-at-once, simulated parents	Wrong plans 0.8% against 2.8%; 1.17 questions against 0.44	Simulator is our assumption
Explainer, 700 held-out questions	80.9% of answers pass every check; 0.4% invented a figure; what a person reads has 100% figure precision	Mostly rewords; see 2.10
Review router, later period it never saw	Puts 63.2% of the flagged payments in its top 1% (random order: 1.2%); average precision 0.437, about 240 times random	Orders well; as a yes/no flag it is modest (section 2.11)

Limits. Providers, fees, rates, KYC, banks and settlement are simulated, and the rates and fees are invented. No real parent or bank has used it, and no outside party has tested its security. The review router was trained on public synthetic data, and as a yes/no flag it is modest. The chain is an in-memory simulation. Most Bangladesh Bank figures are unverified placeholders. Section 6 has the full list.

---

01 / SYSTEM ARCHITECTURE

1. System architecture

1.1 The problem and the design principle

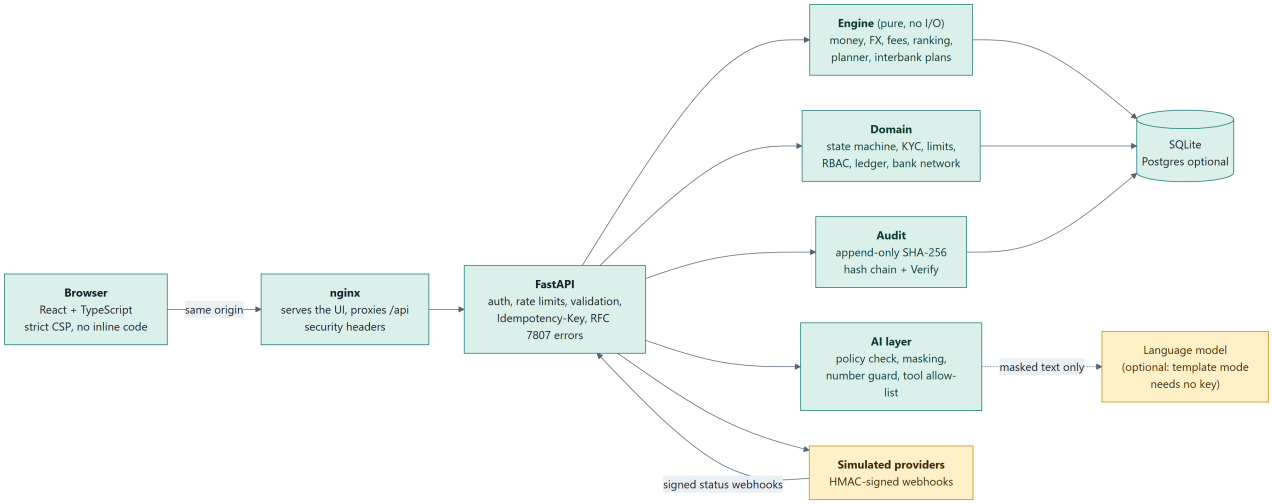
Five pains drive the design, and each maps to a component:

Pain	Where it is handled
The posted fee hides the exchange-rate margin	True-total-cost ranking (2.1)
Fees deducted on the way leave the university short	Shortfall warning and charge-code advice (2.1)
The due date spans two calendars	Deadline planner and deadline fit (2.3, 2.4)
Paperwork and unclear documents	Checklist per corridor, KYC gate, compliance report (1.4, 4.3)
A large transfer approved with little to check it against	Approval summary hash, MFA, bank review desk (1.4, 2.12)

The design principle is four sentences: **the engine calculates, the AI explains, the human approves, and every step is logged.** Each is a mechanism with a test:

Statement	Mechanism	Test
The engine calculates	A pure engine with no database, clock or network access; money as integers; a number guard rejects any figure in a reply that the engine did not produce	`test_engine_quote.py`, `test_ai_number_guard.py`
The AI explains	Models only reword engine drafts or propose a draft; every rewrite passes code checks or the engine's own text is shown	`test_explainer_integration.py`
The human approves	Permission table in code; approval needs a fresh MFA proof bound to the payment and to the summary hash; the AI account gets `AI_CANNOT_APPROVE`	`test_rbac.py`, `test_api_payment_guards.py`
Every step is logged	One audit event per state change in the same transaction; SHA-256 hash chain; triggers block edits	`test_audit.py` and the tamper demonstration

1.2 Components



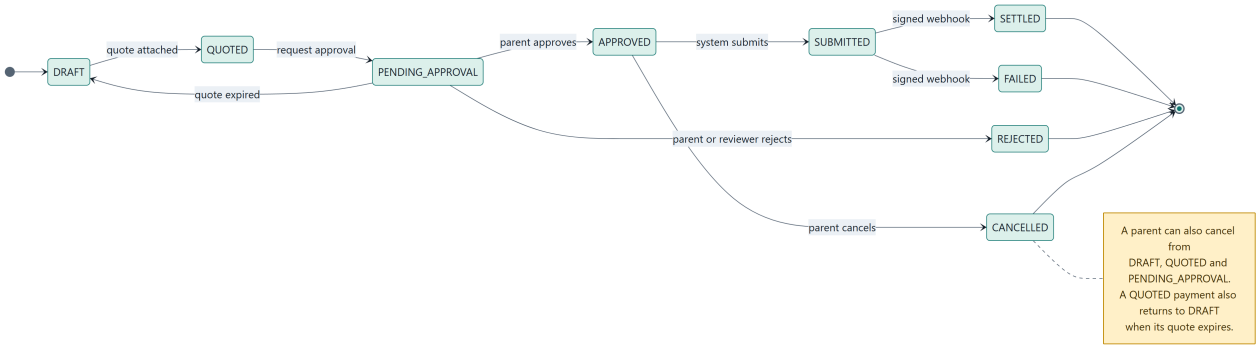

```
Browser (Next.js static export, React + TypeScript)
       |  same origin, strict CSP
       v
    nginx  --- serves the built UI, proxies /api, sets X-Forwarded-For
       |
       v
FastAPI  (security middleware: request id, size limit, headers, X-Sandbox)
  |   auth (JWT ES256) - RBAC - rate limit - schema validation - Idempotency-Key - RFC 7807 errors
  |
  +--> Engine    pure functions: money, FX margin, fees, ranking, planner, runway, MT103 preview
  +--> Domain    state machine, KYC gate, limits, sanctions check, recipients, documents, rules
  +--> AI layer  policy check, facts, masking, models (optional), number guard, tool guard
  +--> Audit     append-only hash chain, verify
  +--> Providers mock adapters A, B, C  -->  simulator  -->  HMAC-signed webhooks back to the API
  +--> Bank network  invented banks, clearing houses and a correspondent  -->  double-entry ledger
  +--> Sandbox chain Solidity contracts on an in-memory EVM: escrow, tokenised deposit, audit anchor
  |
  v
SQLite (Postgres optional), foreign keys on, CHECK constraints, audit triggers
```

Layer	Folder	Responsibility	I/O
Engine	`backend/app/engine/`	Every number: costs, ranking, deadline, runway, student budget, bank-transfer journals	None (pure)
Domain	`backend/app/domain/`	State machine, guards, RBAC, KYC, limits, config, ledger, bank network, bank review	Database through a passed-in session
API	`backend/app/api/`	Routes, authentication, idempotency, rate limits, errors	HTTP
Audit	`backend/app/audit/`	Hash chain and verification	Database
Chain	`backend/app/chain/` , `contracts/`	Compiled contracts, ABI codec, in-memory EVM (`py-evm`)	None (memory)
Providers	`backend/app/providers/`	Mock providers, simulator, webhook signing	HTTP (webhooks)
AI	`backend/app/ai/`	Orchestrator, masking, guards, tools, the four models	Optional call to an external model
Web	`web/`	Dashboard, compare, payments, recipients, documents, student guide, audit, rules, bank network, escrow, bank review, sandbox, assistant	HTTP

1.3 Design decisions and what they cost

Decision	Why	What it costs
Money as integer minor units plus a currency; rates as `Decimal` ; each part rounded once, half-up, at the end	Totals are exact and reproducible; a test checks that the total equals the sum of its parts	More conversion code; display strings are built in the engine
A pure engine	Every number has one origin and can be tested without a database	The domain layer must pass data in
SQLite, with triggers for append-only tables	One-command start (`docker compose up`)	No database roles: an owner can drop the triggers (risks R7, R8)
Small local models (ONNX or NumPy) instead of a hosted model by default	Works offline with no key; no data leaves the machine; each model file's hash is checked	Modest quality, measured and reported in sections 2.8 to 2.11
Every model is optional, with a deterministic fallback	No model failure can block the payment flow; `NLU_MODE` , `DIALOGUE_MODE` , `EXPLAINER_MODE` and `RISK_MODE` can each be switched off	Two paths to test
Every payment needs a person's MFA authorisation, whatever its risk score	The score is a routing aid, not an authority	No automatic approval path, by design
Simulated providers, banks, KYC and chain	No real money, no credentials: a judge can run it	Figures are illustrative and not calibrated against real quotes

1.4 Payment lifecycle




```
DRAFT -> QUOTED -> PENDING_APPROVAL -> APPROVED -> SUBMITTED -> SETTLED
            |            |                |             |-----> FAILED
            |            |-> REJECTED     |-> CANCELLED
            +--- quote expires: QUOTED or PENDING_APPROVAL returns to DRAFT
```

The state machine is a transition table. Every row names the event, the allowed source states, the target state, who may trigger it (parent, AI, admin, system, provider) and the guards that must hold. An illegal move returns `409 INVALID_STATE`. Each transition writes one audit event in the same database transaction, and an optimistic-lock version column stops two simultaneous approvals from both succeeding.

Approval guards: the quote is still valid, KYC is verified, the recipient is allowlisted, the mock sanctions check is clear, the sandbox per-transaction and daily caps hold, the bank review (if one was opened) is cleared, a fresh MFA proof is presented, and the hash of the approval summary the person saw matches.

1.5 Request path

1. The parent compares providers, creates a draft and requests approval. The assistant can explain any number and can *propose* a draft, but a person clicks each step.
2. Requesting approval freezes a summary (recipient, amount, rate, charge code, total cost, latest send date, quote expiry) and stores its SHA-256. The risk router rates the payment and may open a bank review.
3. The parent approves with `X-MFA-Proof` and `confirm_summary_hash`. If anything changed, the hash differs and the approval is refused.
4. A background worker submits the approved payment after a short cancellation window, assigns a UETR and hands it to the mock provider. The simulator later sends an HMAC-signed webhook; the API verifies it and applies `SETTLED` or `FAILED`.

1.6 Data model

`users`, `recipients` (account stored encrypted with AES-256-GCM, only the last four digits shown), `quotes` (inputs, results, mid-rate source and time, input hash), `payments` (state, summary hash, version for optimistic locking), `payment_transitions`, `audit_events`, `audit_head`, `idempotency_keys` (unique per user and key), `webhook_events` (unique event id), `config_rules` (value, source URL, last checked, verified), `kyc_records`, `documents`, `mfa_proofs`, `refresh_tokens`, `provider_ledger`, and for the bank network `bank_accounts`, `bank_transfers`, `bank_transfer_events`, `ledger_journals` and `ledger_entries`. The payment state is limited by a CHECK constraint, and triggers block `UPDATE` and `DELETE` on `audit_events`, `ledger_journals` and `ledger_entries`.

---

02 / CORE ALGORITHMS & TRAINED COMPONENTS

2. Core algorithms and trained components

2.1 True total cost

Every rate is BDT per one unit of foreign currency. A provider selling foreign currency quotes above the mid-market rate, and the gap is the hidden margin. For one provider with target amount `T` (foreign), mid rate `m` and margin `g` percent:

```
provider_rate      = m * (1 + g/100)                          (six decimals)
fx_cost_bdt        = (provider_rate - m) * T
if charge_code == OUR:   sender_extra_bdt = (intermediary_fee + receiving_fee) * m ; shortfall = 0
else (SHA or BEN):       sender_extra_bdt = 0 ; shortfall_foreign = intermediary_fee + receiving_fee
                         (BEN also deducts the provider's fixed fee from the beneficiary)
shortfall_bdt      = shortfall_foreign * m
total_cost_bdt     = fx_cost + fixed_fee + swift_fee + sender_extra + shortfall_bdt
cost_pct           = total_cost_bdt / (T * m) * 100
parent_pays_bdt    = T * provider_rate + fixed_fee + swift_fee + sender_extra
```

We rank by `cost_pct`, then by what the parent pays, then by provider id, so the order is deterministic. We rank by percentage of the amount at the mid rate because the parent's real question is "how much of my money disappears", and a flat fee comparison cannot see the margin. A provider that cannot offer the requested charge code is quoted with its default one, and the result carries both codes and a warning.

Worked example (synthetic; GBP 10,000; mid rate 160.00):

Provider	Rate	Fixed + SWIFT	Charge	Shortfall	Total cost	Cost %	Parent pays
B	161.60	3,000 + 1,500	OUR (GBP 30 intermediary)	0	BDT 25,300	1.58%	BDT 1,625,300
C	162.00	500 + 0	SHA (GBP 25 + 10 deducted)	GBP 35	BDT 26,100	1.63%	BDT 1,620,500
A	163.20	1,500 + 1,000	OUR (fees bundled)	0	BDT 34,500	2.16%	BDT 1,634,500

B ranks first. C ranks second but carries a red shortfall warning: the university would receive GBP 9,965. In B's BDT 25,300, the FX margin is BDT 16,000 (63%), the fixed fee 3,000, the SWIFT fee 1,500 and the OUR fees 4,800. Every figure in this table is asserted in `backend/tests/test_engine_quote.py` (blueprint test T01). The providers and their fees are invented for the sandbox; the table shows how the engine ranks, not what a real provider would charge.

Zero, negative and unsupported amounts are rejected with `422 VALIDATION_FAILED`.

2.2 Quote lifecycle

A quote stores every input, the mid-rate source and timestamp, and an input hash. It expires after 300 seconds (configurable). Approving with an expired quote returns `409 QUOTE_EXPIRED` and sends the payment back to DRAFT; the parent requotes.

2.3 Deadline planner

A business day is a day that is a working day in both Bangladesh (Friday and Saturday off) and the host country (its own weekend and holidays), from configuration.

```
latest_send_date    = due_date stepped back (settlement_days + buffer_days) business days
recommended_send    = latest_send_date stepped back one more business day
earliest_send_today = today if business day and before the bank cut-off, else the next business day
status:  ON_TRACK  if earliest <= recommended
         TIGHT     if earliest <= latest        (using the buffer)
         AT_RISK   if earliest <= latest_without_buffer
         LATE      otherwise
```

Every skipped day is returned with its reason ("BD weekend", "GB holiday: ...") and shown in the interface. A scan limit stops a misconfigured calendar from looping.

2.4 Deadline fit

The cheapest provider can be the slowest, and money that lands after the due date means a late fee. Given the send date and each provider's settlement time in business days, `engine/deadline_fit.py` says which providers arrive in time and which of those is cheapest. If the cheapest would be late, the quote says so and names the next one.

2.5 Smart send plan

Fixed and SWIFT fees are paid on every transfer while the margin scales with the amount, so how often money is sent changes the real cost. For a total `T` over `m` months the engine considers monthly, every three months, every six months and all at once. It splits `T` into equal transfers (amounts differ by at most one minor unit and sum exactly to `T`), prices each transfer with every provider using the formulas of 2.1, and keeps the provider with the lowest total. A provider that would leave the beneficiary short is never chosen while another does not.

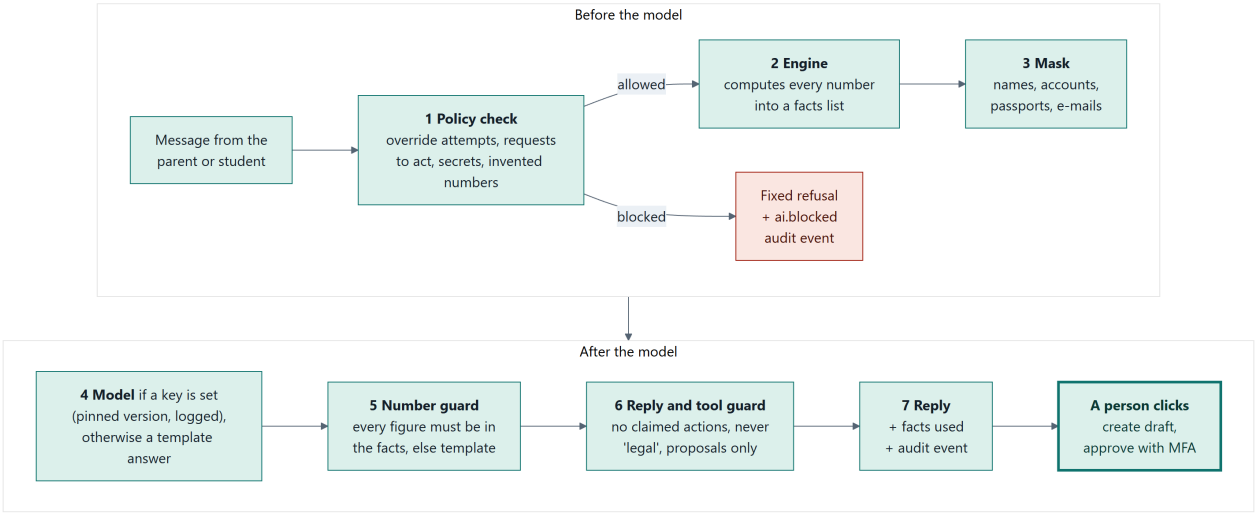
Worked case (synthetic; GBP 12,000 over 12 months; charge code OUR): monthly BDT 68,400 with Provider A; every three months BDT 48,400; every six months BDT 37,800 with Provider B; all at once BDT 28,500, a saving of BDT 39,900 against monthly (asserted in `test_engine_cadence.py`). The engine does not predict exchange rates, so the trade-off (money sent early, rate locked early) is stated and not priced.

2.6 Currency-drop stress test, runway and student budget

The stress test prices a stored quote again with the foreign currency 5, 10, 15 or 20% dearer in taka, for the same provider, using the same fee engine. Fixed fees in taka do not move. It is a what-if on synthetic rates, read-only, and says it is not a forecast. The runway reports the whole months an amount covers, rounded down so it never over-promises. The student budget converts a month's costs in 20 student cities to taka and rounds up, so it never promises a parent less than a month needs. Its city costs are invented and marked "Not verified".

The next six subsections cover the assistant pipeline and the four trained models inside it.

2.7 The assistant pipeline




```
user message --> 1 policy check --> 2 facts from the engine --> 3 mask personal data
             --> 4 model (optional) --> 5 number guard --> 6 tool guard --> 7 un-mask --> 8 audit
```

1. **Policy check.** Pattern rules flag instruction overrides, action requests (approve, send, cancel), limit or configuration changes, privilege claims, secret requests and requests to invent numbers. A flagged message gets a fixed refusal and the model is not called.
2. **Facts.** The orchestrator builds a numbered list of facts from engine output only.
3. **Masking.** Names become `[PERSON_n]`, account numbers and long digit runs become partial tokens, and passports, e-mails and phones are removed and never restored.
4. **Model.** Optional. By default the local trained explainer (2.10); a configured external model replaces it. The model receives masked facts and must answer with the ids of the facts it used.
5. **Number guard.** Every number in the reply must equal a number in the allowed facts. Anything else fails the reply, which is retried once and then replaced by the deterministic template answer built from the same facts.
6. **Tool guard.** The model may only propose `create_draft` or `request_approval`. Approve, submit, cancel, change limit, allowlist and edit configuration are not available to it.
7. **Audit.** One event records the prompt hash, output hash, model version and guard result.

Questions to the person and read-backs of what the assistant understood are shown word for word. We added that rule after the Docker run (section 5) showed the explainer rewording a confirmation question badly.

2.8 The reader: intent and marked words

Problem. Parents write untidy requests ("kindly help me send ten thousand pounds for my son's tuition, last date end of November"). A rule-based reader gets 61.1% of intents right on our frozen hand-written messages.

Design. A small fine-tuned model (`bge-small-en-v1.5` [5], 33.8 MB as int8 ONNX) predicts one of 14 intents and marks the exact words that carry an amount, a currency, a date, a period, a charge code, a purpose or a provider. It only proposes. Plain code (`ai/nlu/normalize.py`) turns the marked words into typed values (integer minor units, a real date) and refuses anything it cannot read exactly. When the rules and the model read a field differently, the field is left empty and the assistant asks. The model's intent is used only at confidence of at least 0.8, and a confident `ATTACK` can only add a block. Three intents (stress test, student costs, arrival guide) are decided by exact rules because the model never saw those questions. The model has no permission of its own.

Training. 35,588 messages, mostly generated from templates, plus real bank-customer questions (Banking77 [2], CC BY 4.0); 6 epochs on an RTX 5060 Ti in 238 seconds. Results come from `ml/evaluate.py` (`submission/evidence/ai_evaluation.json`), on data the model never trained on:

Evaluation set	n	Right intent: rules	Right intent: model	Attacks blocked: rules	Attacks blocked: model
Frozen hand-written holdout (now reproduced)	113	61.1%	**88.5%**	28.6%	**92.9%**
Real bank questions (Banking77 test, mapped)	680	40.1%	**95.1%**	n/a	n/a
Held-out template wording	11,004	62.0%	**90.2%**	52.1%	**85.1%**

On the hand-written set the model also reads a plan request exactly 100% of the time against 57.6% for the rules; harmless messages blocked by mistake are 2.0% for both.

What this does not show. The 113-message set is small (about plus or minus 7 points) and was reproduced several times, so it is not a fresh blind study. The training and generated test data are mostly synthetic. No real parent's messages were used. About one attack in fourteen still gets through the model to a harmless answer. On CPU the reader takes a median of 18 ms and a 95th percentile of 57 ms per message in the last evaluation run. With no model file, a failed integrity check or `NLU_MODE=off`, the assistant runs on the rules alone.

2.9 The dialogue policy (reinforcement learning)

Problem. When a message starts a payment plan, should the assistant plan at once, ask for a missing amount, date or period, or read the values back for a yes? Planning at once is fast but sometimes wrong; asking every time is safe but tiresome.

Design. A 30 → 64 → 64 → 5 network, trained with Double DQN [3] against a simulated parent whose sentences were run through the real reader, so the noise it learns from is the system's own. The state is how each value was read, how sure the model was, whether a second amount was mentioned, and the turn count. Rewards: +1 for a correct plan, +0.5 for a correct deadline or period, -2 for a wrong amount, -1 for a wrong deadline or period, -0.05 per question, -0.5 if the parent gives up. The network never decides what is allowed: `ai/dialogue/state.py` computes the legal actions in plain code and the network only ranks those. None of the five actions moves money.

Results (6,000 simulated conversations per policy; `ai_rl_evaluation.json`):

Policy	Mean return	Questions asked	Wrong plans
Plan as soon as the amount is known (before)	1.164	0.44	2.8%
Random legal action	1.236	1.07	2.2%
Trained policy	**1.358**	**1.17**	**0.8%**
Best hand-written rule (ask everything, confirm every time)	1.343	1.97	0%

The policy matches the best hand-written rule's return with about 40% fewer questions, at the price of 0.8% wrong plans in simulation. Whether that trade is right is a product decision, not a result. The simulated parents and rewards are our assumptions, the policy has never met a real person, and under a harsher wrong-amount penalty it did not quite match "always confirm". The API therefore adds a mandatory read-back envelope that the policy cannot override. `DIALOGUE_MODE=off` plans as soon as the amount is known.

2.10 The explainer: rewording the engine's answer

Problem. Template answers read stiffly, and a free language model can invent a figure. We wanted a model that rewords the engine's draft and is structurally unable to change a number.

Design: verified distillation. A larger open model (Qwen2.5-1.5B-Instruct [6]) proposed rewrites for 6,000 training drafts. A code check, `check_rewrite`, kept only faithful ones: no new figure, most figures kept, the disclaimer and any refusal kept, no action or legal claim, "(synthetic)" kept. More than half of the raw teacher output failed (7,785 of 18,000

proposals passed, 43.3%), which is why the student never sees a raw proposal. A small student (`flan-t5-small` [7], 77M parameters) was fine-tuned on the verified rewrites in 12 minutes. The vocabulary was pruned from 32,000 to 5,124 pieces and the model exported to int8 ONNX (53 MB). At run time every rewrite is checked again; a failing rewrite is thrown away and the engine's own text is shown.

Results on 700 held-out questions (`ai_explainer_evaluation.json`):

	Trained explainer	Same model, untrained
Answers that pass every check and are shown	**80.9%**	20.1%
Answers that invented a figure	**0.4%** (3 of 700)	1.4%
Answers identical to the engine's draft	48.6%	0.4%
Figure precision, raw output / what a person reads	99.8% / **100%**	96.1% (raw)
Figure recall, raw output / what a person reads	97.5% / 98.8%	21.3% (raw)

What this does not show. It mostly paraphrases: about half of its accepted answers equal the draft. It cannot finish long structured answers (stress tests 0% shown, arrival guides 21%), so drafts over 120 words skip the model. The checks look at figures, disclaimers and claims, not at style, so it can word an answer clumsily. It was trained and tested on synthetic questions. Its value is that the safe path costs nothing: the engine's answer is always one failed check away.

2.11 The review router: which payments a person looks at first

Problem. A bank reviewer has limited time. Which payments deserve a look first, and why?

Data. IBM's public synthetic anti-money-laundering transactions [1] (HI-Small: 5,078,345 transactions, 5,177 labelled by the generator; CDLA-Sharing-1.0). The split is by time: the first 60% trains, the next 20% tunes, the last 20% (1,015,564 transactions, 1,797 flagged) is the test. Every count of "earlier payments" uses only earlier rows.

Design. A small network on eight inputs in four factors: **amount** (a percentile within the currency), **frequency** (payments by this household in the last day, the last week and ever), **timeline** (time since the last one) and **person** (whether this household has paid this receiver before). Bands come from cut-offs set on the tuning period only: `HIGH` is the top 0.1% of scores and `MEDIUM` the next 0.4%. For each payment the model hides each group of inputs in turn to show which factors drove the score. A `MEDIUM` or `HIGH` rating opens a bank review; `LOW` takes the fast lane. The model never clears, declines or approves anything, and every payment still needs the parent's MFA authorisation.

What we left out, and how we found out. The first version also saw the payment channel, the currencies and the hour, and scored higher. Counting labels per channel (`ml/aml/ablate.py`) showed that in this data no wire and no cross-currency transfer is ever labelled as laundering. A model with those inputs would learn that a cross-currency wire, which is what every StudyRoute payment is, is the safest thing there is. That is a habit of the generator, not a fact about money, so those inputs were removed; dropping the channel inputs cost nothing in average precision (0.449 without, 0.412 with, first pass).

Results on the test period (1,015,564 transactions, 1,797 of them flagged; `ai_risk_training.json` and `ai_risk_operating_points.json`).

Ordering. How many of the flagged payments are among the 1% of payments with the highest score?

Ordering	Flagged payments found in the top 1%	Average precision
Random order	1.2%	0.0018
Shipped model	**63.2%**	**0.437** (about 240 times random)

As a yes/no flag. The desk sends a payment to a person when its band is medium or high. Accuracy is the share of all payments handled correctly (a flagged one sent to a person, an unflagged one left alone).

Routing	Sent to a person	Accuracy	Precision	Recall
Medium or high (the desk's rule)	3,474 of 1,015,564 (0.34%)	99.68%	29.2%	56.5%
High only	1,548 (0.15%)	99.83%	52.6%	45.4%
Best F1, threshold from the tuning period	962	99.86%	67.5%	36.1%
Never flag anything	0	99.82%	–	–

99.82% of the payments are not flagged, so never sending anything to a person is already 99.82% accurate: accuracy alone cannot tell this router from no router, the ordering table can. Precision and recall cannot both exceed 90% on this data at any threshold; we did not change the evaluation set or threshold to get a bigger number.

Model comparison (`docs/AI_RISK_MODELS.md` ; 24 configurations: ten learned models and four rule or random baselines, on the eight shipped inputs or thirteen):

Configuration	Tuning AP	Test AP
XGBoost, 13 inputs	**0.488**	0.448
Random forest, 13 inputs	0.475	0.540
Neural network 128-64, 13 inputs	0.438	**0.545**
Neural network 128-64, 8 inputs (shipped)	0.340	0.437
Logistic regression, 8 inputs	0.285	0.088
Rule: a new receiver	0.073	0.114
Random order	0.001	0.002

By our own rule (choose on the tuning period only) XGBoost on thirteen inputs ranks first, yet on the test period it is no better than the shipped network within noise (0.448 against 0.437); neighbouring models differ by about 0.02 average precision, which is within noise with 1,797 flagged transactions. The thirteen-input variants score higher than the shipped network on both periods. We did not adopt them before submission: they need five more inputs computed from payment history at run time, and the review desk would have needed re-testing. It is the first improvement on our list.

2.12 The bank review desk

When approval is requested, a `MEDIUM` or `HIGH` rating (or a sandbox name-list match) opens a `bank_reviews` row with a target time, the signals and the model's reading, and writes one audit event. The approval guard then refuses `BANK_REVIEW_PENDING` until a reviewer answers. The reviewer, an admin user, answers `CLEAR` , `ASK_INFO` or `DECLINE` with a single-use MFA proof under the scope `review_payment` . A decline calls the existing rejection, so the state machine is unchanged. The parent's short reply is stored sealed with the field cipher. A clock (on time, due soon, overdue) and a reminder with a cool-down make slow reviews visible. Plain rules (a large amount, several payments that add up to one, a new receiver, no fee demand note) are context for the reviewer and decide only when the model is not loaded. See `docs/BANK_REVIEW.md` .

---

03 / SECURITY DESIGN

3. Security design

The full threat and control matrix, with the test that proves each row, is in `docs/SECURITY.md` . The permission tiers and the 55 known risks are in `docs/SECURITY_SELF_ASSESSMENT.md` .

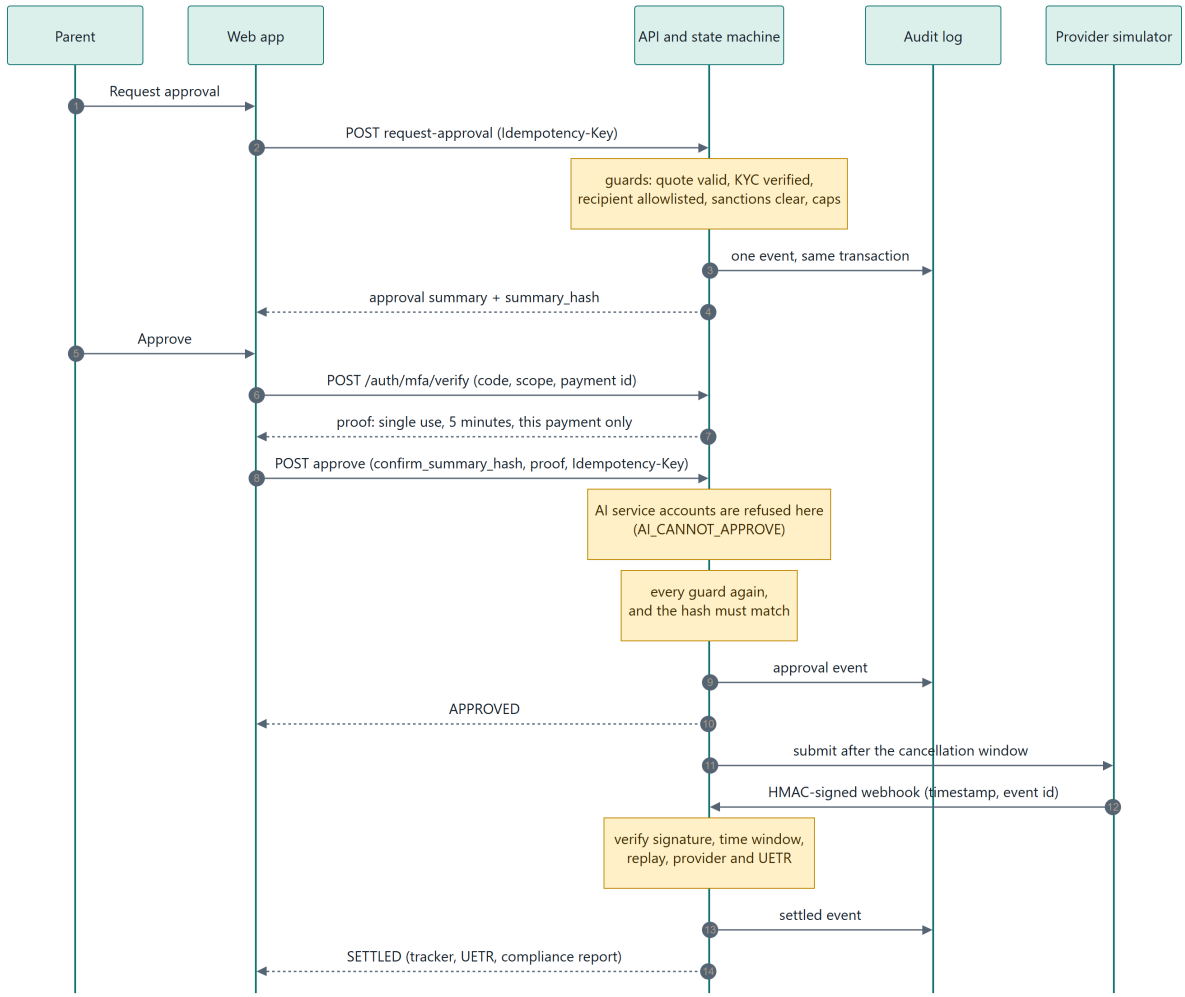
3.1 Identity and sessions

Passwords use Argon2id with the OWASP minimum parameters. Access tokens are ES256 JWTs valid for 15 minutes. The programmatic API issues rotating refresh tokens with reuse detection. The browser uses a separate path: an opaque `HttpOnly`, `SameSite=Strict` cookie session, stored server-side as a hash and bound to the access token. Browser authentication endpoints require an exact trusted `Origin` and a custom header, so a cookie alone is never API authority. The access token lives only in page memory. Repeated failures lock the account with the same answer for unknown users, so e-mail addresses cannot be discovered by trying to lock them.

3.2 Authorisation

Effective authorisation needs the role's permission, a matching scope in the token, and ownership of the object; a request for another household's object gets `404`. The AI service account has no approve, cancel, reject, allowlist, limit or configuration permission, and tests check both the permission table and the AI account's refusals (`test_rbac.py`, `test_api_ai_rbac.py`).

3.3 Payment safeguards



MFA step-up. A fresh proof is required to approve a payment, allowlist a recipient, confirm an escrow, make a bank transfer and answer a bank review. It is valid 5 minutes, single use, and bound to the user, the scope and the object id, so a proof for one payment cannot approve another. TOTP (RFC 6238 [4]) works in every mode; sandbox mode also accepts a documented fixed code so judges need no authenticator app. That fixed code is a sandbox shortcut, not production-strength verification.

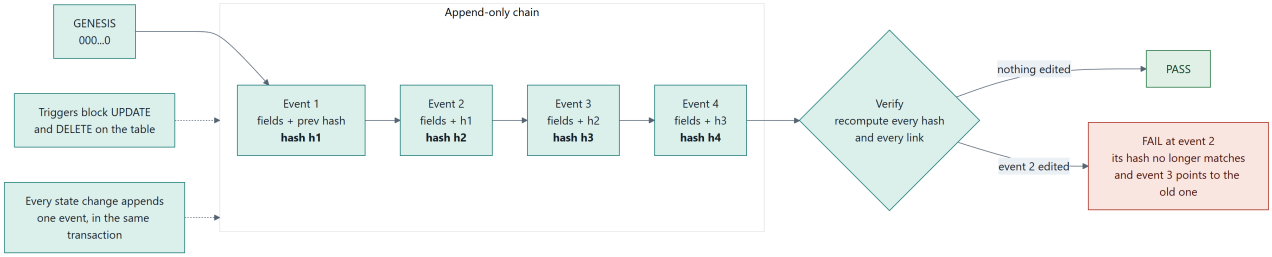
Idempotency. Every payment write needs an `Idempotency-Key`. The key, a hash of method, path and body, and the response are stored in the same transaction as the change, for 24 hours. The same key and body replays the stored response with no second effect; the same key with a different body returns `422`. A unique constraint makes two simultaneous requests safe.

Webhooks.

```
signature = HMAC_SHA256(secret, timestamp + "." + raw_body)      header: t=<ts>,v1=<hex>
```

The receiver reads the raw body, rejects a timestamp more than 300 seconds old, recomputes the HMAC for the current and previous secret, compares in constant time, de-duplicates by event id, and only then parses the JSON. Each rejection writes an audit event and changes nothing.

3.4 The audit log: what it guarantees and where it stops




```
GENESIS             = "0" * 64
canonical(e)        = JSON of e without this_event_hash, keys sorted, no whitespace, UTF-8
this_event_hash(e)  = SHA256( prev_event_hash + canonical(e) )
verify:  prev = GENESIS
         for e in events ordered by seq:
             fail at e.seq if e.prev_event_hash != prev
             fail at e.seq if SHA256(prev + canonical(e)) != e.this_event_hash
             prev = e.this_event_hash
```

Appending locks the head row, so concurrent writers keep one linear chain, and triggers block `UPDATE` and `DELETE`. In a recorded run, editing one amount in event 74 made verification fail at that event, and restoring it passed again (`submission/evidence/tamper_demo.json`).

The chain is tamper-evident, not tamper-proof. The audit events and the simulated chain's log share one SQLite file, so someone who owns the file can rewrite and re-hash both consistently; a regression test demonstrates exactly this. What catches it is a checkpoint stored outside the database (`scripts/audit_checkpoint.py`). The simulated chain adds a second anchor and an on-chain exploration, not independent security: it has no peers, no consensus and no keys.

3.5 Data and AI safety controls

Account numbers and document text are stored with AES-256-GCM, and only the last four digits are shown. Log output passes a redaction filter. Personal data is masked before any text reaches a model. The number guard, tool guard and policy check of section 2.7 apply to every model, local or external. Each model file's SHA-256 is checked before it loads, which establishes consistency with the manifest, not the publisher's identity.

3.6 Web and abuse controls

Strict content-security policy with no inline style and no inline script except the exported pages' own bootstrap scripts, allowed by SHA-256 hash; `nosniff`; `no-referrer`; frame-ancestors none; HSTS; exact-origin CORS without credentials. Per-IP and per-user rate limits, a request size limit, and schema validation that rejects unknown fields.

3.7 Compliance posture

Rules and limits live in configuration with a `source_url`, a last-checked date and a `verified` flag. Anything unchecked shows a yellow "Not verified" badge and the sentence "Indicative. Confirm with your AD bank." The application never says that something is legal or within a limit. On 18 October 2026 we read section 10 of Bangladesh Bank's circular FEPD-1 No. 40 (30 September 2026) and marked four study-abroad rules verified: one AD bank branch for the whole course, one academic year at a time, release per the institution's estimate, and international cards for education payments. The USD 2,000 a month living-cost figure from a secondary source is not in that text, so it stays unverified. This is our reading and no lawyer or AD bank has reviewed it.

3.8 Residual risks and release gates

The ten residual risks that matter most, from the separately prepared Security Self-Assessment (edition 1.1), which covers the same commit and cross-references our 55-item register in `docs/SECURITY_SELF_ASSESSMENT.md`:

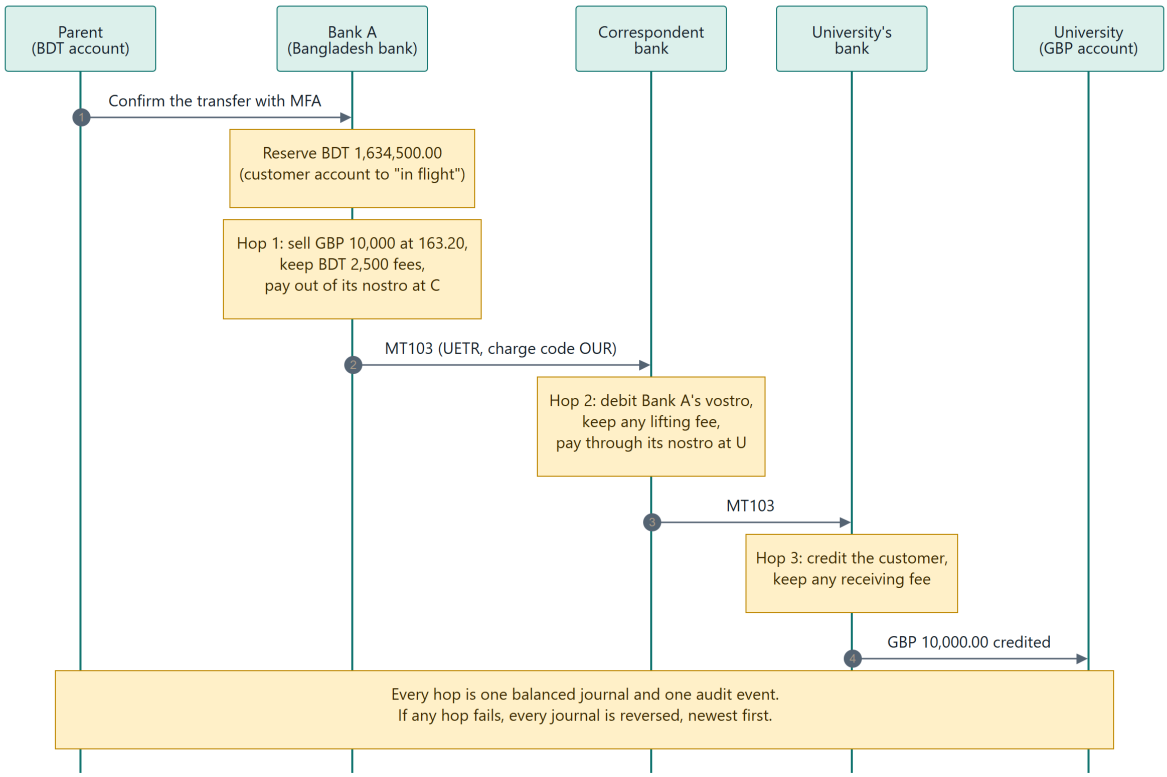
ID	Residual risk	Priority
SR-01	Shared demo accounts, public hints and a fixed MFA code: deliberate sandbox shortcuts	High
SR-02	Credentials exposed in earlier workspace material; revocation is the owner's to do	High
SR-03	A privileged database owner can rewrite the audit log and the simulated-chain log consistently	High
SR-04	Invented rails and quotes, and most regulatory thresholds, are not production rules	High
SR-05	A public deployment (TLS, proxy trust, restore) has not been validated	High
SR-06	Heuristic filters and statistical models miss some attacks	Medium
SR-07	Model and dependency substitution: a hash shows consistency, not the publisher's identity	Medium
SR-08	Privacy, retention and external AI providers	Medium
SR-09	Process-local rate limits; no identity lifecycle at scale	Medium
SR-10	The demo reviewer sees every household; there are no real mandates or dual control	Medium

Eight gates stand between this sandbox and any real-money or real-person use: secret custody, real identity and access, HTTPS and ingress, persistence and restore, independently held integrity checkpoints, supply-chain checks, licensed real-world integration, and independent assurance. None is established.

04 / THE SIMULATED FINANCIAL LAYER

4. The simulated financial layer

4.1 The bank network



`engine/interbank.py` turns a transfer request into a plan: a hold journal and an ordered list of steps, each with the institution that acts, the simulated message and a balanced journal. For every currency, debits must equal credits or building the journal raises. National transfers run over a same-bank, instant, next-batch or high-value rail. International transfers start from a Bangladesh bank in BDT and are built from the fee engine's result, so the customer's debit equals what the parent pays and the beneficiary's credit equals what the beneficiary receives. `domain/ledger.py` applies journals, refuses any that would overdraw an account, and unwinds a failed transfer by posting the exact opposite of every journal, newest first. Three checks hold at any moment: the trial balance, the mirror accounts (each nostro equals its vostro) and the entry sums. The configuration is invented. See `docs/BANK_NETWORK.md`.

4.2 Smart escrow and the audit anchor

Three Solidity contracts (`contracts/src/`) are compiled once, with the artifacts committed, and run on `py-evm`, a Python implementation of the EVM. One transaction is one block, a revert changes nothing, and wallets are addresses derived from labels, so there is no key anywhere. The history is kept in an append-only, hash-chained `chain_log` and replayed to rebuild the chain. A parent locks an escrow with a fresh MFA proof for that escrow (three transactions, one audit event); a worker pays each tranche on its date; paying early or cancelling needs MFA again; the reviewer can only freeze or unfreeze. The `AuditAnchor` contract stores audit head hashes, and `GET /chain/verify` reports `MISMATCH` when the database no longer agrees. It is a simulation, not a network. See `docs/SMART_ESCROW.md`.

4.3 The compliance report

For one payment the report collects the originator and beneficiary, ten controls with a result and the evidence for each, the approval proof, the state timeline, the payment's slice of the audit log, and the verification of the whole chain. It

ends with the SHA-256 of its canonical JSON. The web screen recomputes that hash in the browser with the same sorted-key compact JSON as the backend; a test checks the two agree byte for byte, including Bengali text.

---

05 / VERIFICATION & EVIDENCE

5. Verification and evidence

Layer	What was done	Result	Where
Automated tests	One or more tests for each of 24 blueprint scenarios; a test fails if one is missing	1,679 backend tests (1,678 pass, 1 skipped on Windows); 463 web tests; 24 browser tests	`docs/TEST_CASES.md`
Demo flow	Fourteen-step run against a live API	66 checks OK, none failed	`submission/evidence/sandbox_operation_log.txt`
Black-box attacks	Forged tokens, cross-household access, injection, hostile input, webhooks, the bank review desk	146 of 146 defended	`security_2026-10-18/attack_probe.txt`
Other probes	Brute force, data-leak scan, man in the middle, hostile prompts, real Chrome	8/8, all clear (50 checks), 14/14, 42/42, 48/48	`security_2026-10-18/`
Docker	Ubuntu 24.04 VM; smoke, demo, bank review demo, browser audit, hardened overlay	All passed after one fix (below)	`docker_2026-10-18/`
Continuous integration	Six jobs: docker, contracts, web, secret-scan, backend, dependency-audit	All succeeded on the final commit	GitHub Actions

A defect the verification found. The first Docker run failed three demo checks, and the same step had failed four in CI. The trained explainer is on by default in a container and had never run in one before. It reworded the assistant's questions and read-backs badly, and the demo's checks assumed a rewrite is always accepted. We fixed both: questions and read-backs are now shown word for word, and the checks look at the engine's figures instead of exact words. The failing run is kept as evidence.

A reproducibility gap the verification found. One test module, the explainer quality floor, needs the file `backend/tests/data/explainer_cases.json`. A rule in `.gitignore` excludes it, so a fresh clone collects 1,674 tests and skips that module. The 1,679 figure is for a machine that has the file. Adding it to the repository is on our list.

What the evidence does not show. The attack tools were written by the team that wrote the application; no outside penetration test has been done. No real user has used the system. The rates and fees are not compared with real quotes. The Docker run used the archive of an earlier commit with the two fixed files copied in. The trained models were measured on synthetic or mapped data. A separately prepared execution evidence report repeats the backend run on Linux (1,672 passed on the downloaded main archive) and recomputes the audit chain outside the application.

To reproduce: `cd backend && pytest -q`; start the API and run `python -m app.scripts.demo_flow`; run `python ml/evaluate.py` for the reader. Retraining commands are in `docs/AI_AGENT.md`, `docs/AI_RL.md`, `docs/AI_EXPLAINER.md` and `docs/AI_RISK.md`.

---

06 / LIMITATIONS & NEXT STEPS

6. Limitations and next steps

1. **Everything financial is simulated.** Providers, fees, margins, KYC, sanctions screening, banks and settlement are invented. Step one is to compare with real dated quotes for one corridor and publish the gap.

2. **No real users and no partner.** The one-sentence flow has not been timed against a spreadsheet with real parents. A pilot with one bank branch and a few dozen families would test it.
3. **Models are small and trained on synthetic data.** The reader's blind set is small. The policy learned from a simulator of our own making. The explainer mostly paraphrases. As a yes/no flag the router is modest. The thirteen-input variants should be adopted and re-tested.
4. **Security assurance is self-made.** An independent penetration test is the largest missing item. The audit chain needs an externally held checkpoint in any real deployment. Rate limits are per process. The fixed sandbox MFA code must go.
5. **Rules.** Four study-abroad rules are anchored to Bangladesh Bank's circular; the rest are placeholders. Moon-dependent holidays are missing.
6. **Not built.** Credit building, named in Topic C. The escrow and chain remain a simulation; a real version needs a permissioned chain and a bank-issued token.

---

07 / PROVENANCE, LICENCES & AI ASSISTANCE

7. Provenance, licences and AI assistance

Large parts of the code, tests and documents were produced with AI coding assistants working under the team's direction; the repair release is recorded in `docs/PROVENANCE.md`. This report does not itemise individual members' contributions.

Component	Licence	Use
BAAI/bge-small-en-v1.5	MIT	Base of the reader (shipped as int8 ONNX)
Banking77 (PolyAI)	CC BY 4.0	Real customer questions for training and testing the reader
google/flan-t5-small	Apache-2.0	Base of the explainer (shipped as int8 ONNX)
Qwen2.5-1.5B-Instruct	Apache-2.0	Teacher for the explainer; development only, not shipped
IBM AML transactions, HI-Small	CDLA-Sharing-1.0	Trained the review router; development only, data not in the repository
py-evm, onnxruntime	MIT	Run-time dependencies

The full inventory is in `docs/THIRD_PARTY.md`. Market figures in the summary are from The Observer (24 July 2026, citing Bangladesh Bank); the sums and the limits of that business case are in `docs/BUSINESS_CASE.md`.

References

1. Altman, E., Blanuša, J., von Niederhäusern, L., Egressy, B., Luksic, A., Bitar, K. "Realistic Synthetic Financial Transactions for Anti-Money Laundering Models." NeurIPS 2023, Datasets and Benchmarks Track.
2. Casanueva, I., Temčinas, T., Gerz, D., Henderson, M., Vulić, I. "Efficient Intent Detection with Dual Sentence Encoders." 2020 (PolyAI; the Banking77 data set, CC BY 4.0).
3. van Hasselt, H., Guez, A., Silver, D. "Deep Reinforcement Learning with Double Q-learning." AAAI 2016.
4. M'Raihi, D., Machani, S., Pei, M., Rydell, J. "TOTP: Time-Based One-Time Password Algorithm." RFC 6238, 2011.
5. Xiao, S., Liu, Z., Zhang, P., Muennighoff, N. "C-Pack: Packaged Resources To Advance General Chinese Embedding." 2023 (the BGE embedding models, from which `bge-small-en-v1.5` comes).
6. Qwen Team. "Qwen2.5 Technical Report." 2024.
7. Chung, H. W. et al. "Scaling Instruction-Finetuned Language Models." 2022 (the FLAN-T5 models).
8. Observer (Dhaka), "Overseas Education Boom Drives Record Foreign Currency Outflow," 24 July 2026, citing Bangladesh Bank data.